

Event-Driven Traffic Congestion Severity Prediction and Resource Recommendation System

Aman Kumar (2023IMT-010)

Sahil Pal (2023IMT-069)

Vishwajit Sarak Patil (2023IMT-071)

Supervisor: Dr. Saswata Roy

Department of Information Technology
ABV-Indian Institute of Information Technology and Management Gwalior

B.Tech Project: Final Evaluation Report



Outline

- 1 Motivation
- 2 Problem Background
- 3 The Labelling Problem
- 4 Objectives
- 5 Dataset and Features
- 6 Model Architecture
- 7 Stacked Generalization
- 8 Evaluation Methodology
- 9 Results
- 10 Recommendation Engine
- 11 Deployment
- 12 Limitations and Future Scope
- 13 Conclusion

Motivation: Why Event Severity?

- ▶ **Non-recurring congestion** (accidents, breakdowns, public events) is the disruptive kind, and the hardest to plan for.
- ▶ When an event is first reported, the control room must decide **how much to send** (officers, barricades, diversions) **before anyone has seen the site**.
- ▶ That decision is currently made on **experience alone**.
- ▶ Under- or over-deploying both carry real cost: congestion on one side, wasted manpower on the other.

Operational impact

- ✓ Faster first response
- ✓ Right-sized deployment
- ✓ Consistent decisions across shifts

Dataset at a glance

8,173 Bengaluru traffic events
46 raw attributes → 30 engineered features
4 severity classes

What the literature does well:

- ▶ Gradient-boosted trees perform consistently well on **tabular** incident-severity data.
- ▶ Deep learning for traffic congestion is mature, but focused on *recurring*, sensor-driven flow.
- ▶ Stacked ensembles and imbalance-aware evaluation are each well established.

What is missing:

- ▶ These three threads are rarely combined in one pipeline.
- ▶ Severity prediction almost never produces an *operational recommendation*.

Positioning of this work

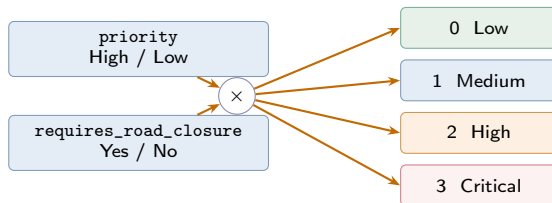
A stacked ensemble of tree-based *and* neural classifiers applied to event-driven severity, evaluated with **front-loaded** attention to class imbalance, and coupled directly to a deterministic resource-recommendation engine.

Challenge

Severity is **not recorded anywhere** in the raw log. The target must be constructed, carefully.

Constructing a Target That Does Not Exist

No severity column exists in the raw log. The target is built by crossing two fields the control room records *independently*, for its own purposes.



Code	Label	Count	Share
0	Low	2,764	33.8%
1	Medium	379	4.6%
2	High	4,733	57.9%
3	Critical	297	3.6%

Low and High are 91.7% of all events.

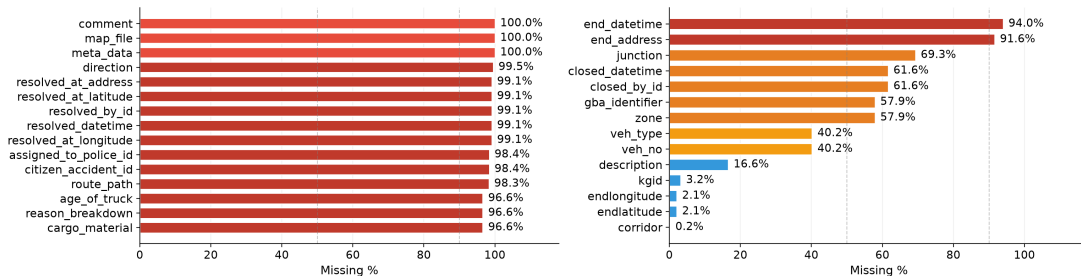
Both source fields are then *removed*

If left in the feature set, a model would simply re-learn this deterministic mapping instead of inferring severity from spatial, temporal and contextual signals, and would be useless on a genuinely new, unassessed event.

- 1 **Construct a four-class severity target** that the raw event log does not contain, without leaking the fields it was built from.
- 2 **Engineer 30 spatial, temporal and contextual features**, including an unsupervised K-Means clustering of event coordinates.
- 3 **Train four classifiers and stack them** out-of-fold with a LightGBM meta-learner, comparing all five honestly.
- 4 **Evaluate for class imbalance**, then convert the prediction into a resource recommendation and deploy it as a working service.

Dataset and Missing-Value Audit

Missing Value Distribution across the 46 raw columns (n = 8,173 rows)



Decision

Columns

Dropped (96–100% missing)

comment, map_file, meta_data, resolution and truck fields

Kept as “unknown” (40–70%)

junction, closed_datetime, zone, veh_type

Retained (0% missing)

coordinates, event cause, priority, closure flag

Dropped (identifiers)

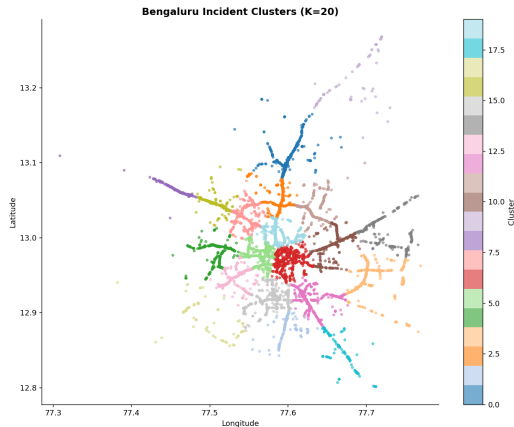
id, client_id, created_by_id

Feature Engineering: 30 Features

Group	Features
Spatial	latitude, longitude, corridor, police station, zone, junction, location cluster
Temporal	hour, minute, day of week, month, day, weekend flag, morning- and evening-rush flags, time-of-day bucket
Contextual	event type, event cause, vehicle type, duration, status, weather flag

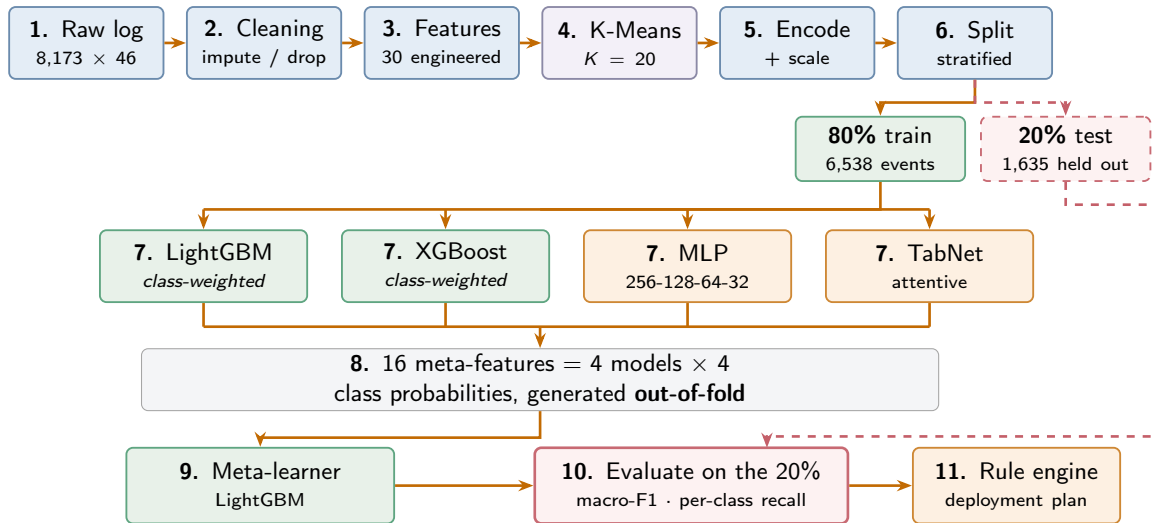
Encoding and splitting

Nine categorical columns label-encoded, numerics standardised. Both transformers are **serialized**, so a new event is encoded exactly as in training. Split 6,538 / 1,635, **stratified**.



K-Means, $K = 20$: unsupervised grouping of the 8,173 event coordinates; the cluster ID becomes feature 30.

Pipeline Overview: End-to-End Flow

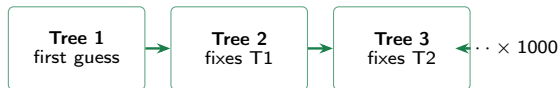


Dashed red = the held-out 20%, never seen during training.

Base Learners: Two Very Different Styles

Gradient-boosted trees: LightGBM and XGBoost

Hundreds of shallow decision trees built *in sequence*, each one specialising in the mistakes the previous trees made.

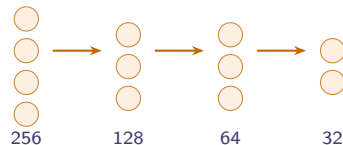


Configuration

1000 estimators, learning rate 0.03. LightGBM grows leaf-wise (63 leaves), XGBoost depth-wise (depth 6). **Both use class-balanced sample weighting.**

Neural approaches: MLP and TabNet

Layers of neurons pass weighted signals forward; the weights are adjusted over many passes to reduce error.



MLP hidden layers. TabNet instead uses **attention** over 5 decision steps.

Class weighting: the deciding choice

$$w_c = \frac{n}{C n_c} \frac{w_{\text{Critical}}}{w_{\text{High}}} \approx 16$$

Applied to the two tree models, **not** the neural ones.

Stacking Without Leakage

The problem

A model that has **already seen** an event predicts it far too confidently. Training the meta-learner on those inflated probabilities teaches it to trust an accuracy that **will not exist** on real new events.

Fold 1	predict	train	train	train	train
Fold 2	train	predict	train	train	train
Fold 3	train	train	predict	train	train
Fold 4	train	train	train	predict	train
Fold 5	train	train	train	train	predict

In each fold, fresh copies of **all four** base models train on the blue slices and predict the orange one.

Out-of-fold matrix

$$z_i = [p_i^{(1)} \parallel p_i^{(2)} \parallel p_i^{(3)} \parallel p_i^{(4)}] \in \mathbb{R}^{16}$$

$p_i^{(m)}$ is model m 's class-probability vector for event i , from the fold that **excluded** i . Stacked: $6,538 \times 16$.

Meta-learner

A LightGBM trained on those 16 columns, learning how much to trust each base model rather than averaging them naively.

Why Accuracy Is the Wrong Headline Metric

	Predicted Critical	Predicted Not Critical
Actually Critical (59)	2 caught (TP)	57 missed (FN)
Actually Not (1,576)	0 false alarms (FP)	1,576 correctly calm (TN)

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{2}{2} = 1.00$$

Of what it *called* Critical, how much was?

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{2}{59} = 0.03$$

Of what *truly was* Critical, how much did it catch?

The trap: MLP on the Critical class

92.4% accurate, 3% Critical recall. 91.7% of events are Low or High, so a model can lean on those two and still be blind to the class the system exists to catch.

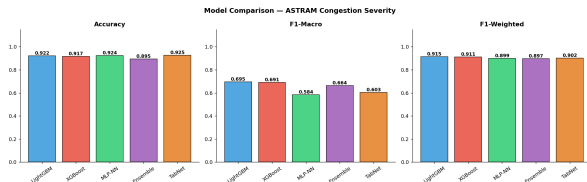
Therefore: macro-F1, not accuracy

$$F1_c = \frac{2P_c R_c}{P_c + R_c}, \quad \text{macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c$$

Every class counts equally, so ignoring Critical is punished.

Overall Results on 1,635 Held-Out Events

Model	Acc.	Macro-F1	Wtd-F1
LightGBM	0.9223	0.6949	0.9148
XGBoost	0.9168	0.6906	0.9114
MLP-NN	0.9242	0.5843	0.8992
TabNet	0.9254	0.6027	0.9020
Stacked Ensemble	0.8948	0.6639	0.8968



A narrow accuracy band

Every model sits between 89% and 93% accuracy, a spread that looks unremarkable. Macro-F1 ranges from 0.5843 to 0.6949, a gap far too wide to be explained by those near-identical accuracies.

The reversal

TabNet posts the **highest accuracy**. LightGBM has the **best macro-F1**. Ranking these models by accuracy would point a deployment decision at the wrong model.

Per-Class Results: Where the Spread Comes From

Critical class (support = 59)

Model	Prec.	Recall	F1
LightGBM	0.54	0.32	0.40
XGBoost	0.46	0.36	0.40
MLP-NN	1.00	0.03	0.07
TabNet	0.75	0.10	0.18
Stacked Ensemble	0.28	0.34	0.31

Medium class (support = 76)

Model	Prec.	Recall	F1
LightGBM	0.58	0.39	0.47
XGBoost	0.56	0.39	0.46
MLP-NN	0.75	0.24	0.36
TabNet	0.83	0.20	0.32
Stacked Ensemble	0.49	0.46	0.47

Key finding

The only two models with *usable* Critical recall, LightGBM (0.32) and XGBoost (0.36), are **exactly the two trained with class-balanced weighting**.

Majority classes are easy

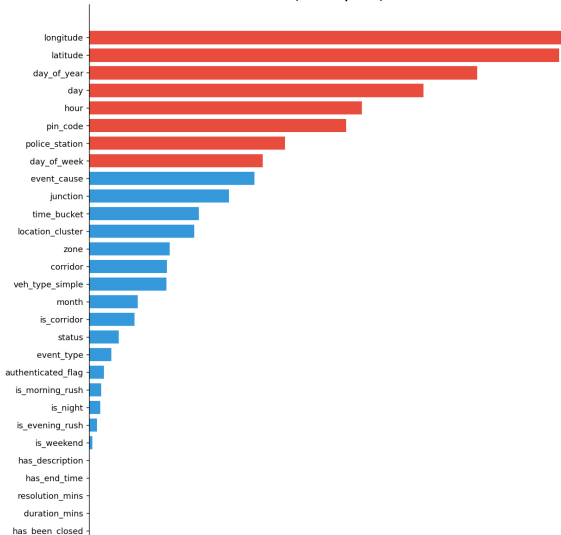
Every model handles Low (F1 0.93–0.94) and High (F1 0.95–0.97) comfortably. The entire macro-F1 spread comes from the two rare classes exactly what accuracy hides.

Honest note on stacking

The ensemble posts the *lowest* accuracy of the five (0.8948). Its in-fold models were lighter, and each fold holds under 100 rare-class examples. It still reaches the second-best Critical recall.

Feature Importances

LightGBM Feature Importances
(red = top 25%)



What ranks highest

- ▶ **Raw coordinates dominate:** longitude and latitude are the two most-used split features.
- ▶ **Fine-grained time next:** day_of_year, day, hour.
- ▶ **Then jurisdiction and cause:** pin_code, police_station, day_of_week, event_cause.

Caveat

Split-count importance favours **high-cardinality** features. That is why the engineered binary flags (is_corridor, is_weekend, rush-hour) sit at the bottom: it counts how *often* a feature is split on, not the size or direction of its effect.

From Severity to a Deployment Plan

Separation of concerns

The models answer *how severe*; the rule engine answers *what to do*. Deterministic and separate, so it stays auditable and tunable **without retraining**.

Severity	Officers	Delay	Barricading	Diversion
Low	2–4	15 min	Cones and indicators	None
Medium	4–8	30 min	Partial, 1 lane + marshal	Advisory alternate route
High	8–14	60 min	Full, both sides + 2 marshals	Mandatory + signage
Critical	15–25	90 min	Corridor lockdown + chain	Police-escorted + advisory

Contextual multipliers

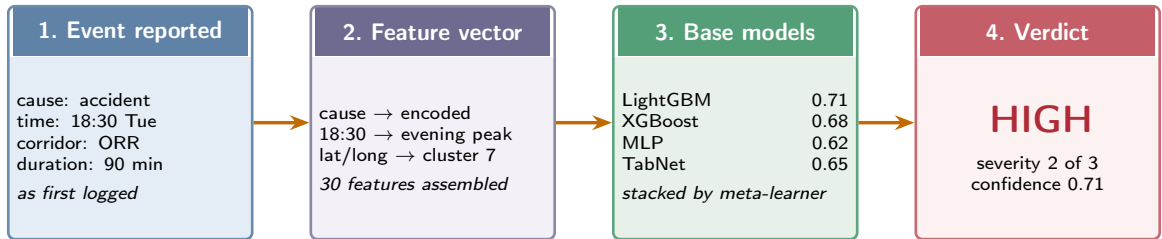
$$n = \lfloor n_{\text{base}} 1.4^c \rfloor 1.25^p$$

$c, p \in \{0, 1\}$: corridor, peak (7–10, 17–21).

Cause-specific actions

accident $\geq$ High: ambulance lane open, trauma centre on standby. public_event $\geq$ High: crowd-control perimeter. Pre-announced: deploy 2 h early.

Worked Example: One Event End to End



5. Rule engine (deterministic)

Base for High: 8-14

$\lfloor 8(1.4) \rfloor = 11$, $\lfloor 11(1.25) \rfloor = 13$

$\lfloor 14(1.4) \rfloor = 19$, $\lfloor 19(1.25) \rfloor = 23$

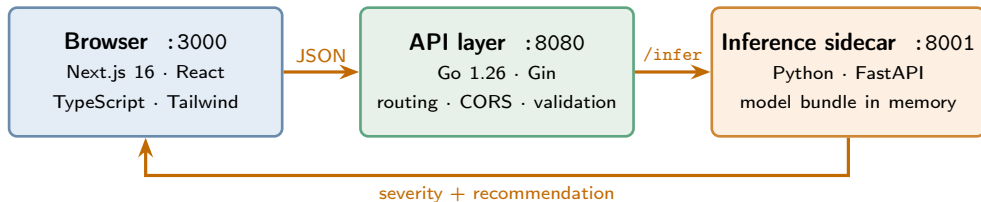
⇒ **13-23 officers**

Recommendation returned

60 min delay; full barricade both sides, 2 marshal points; mandatory alternate route with signage; ambulance lane kept open.

Rule-engine figures are exact; the class probabilities shown are illustrative.

System Architecture and Request Flow



Request lifecycle for POST /api/predict

- 1 Browser posts the event attributes
- 2 Gin handler validates and binds the JSON
- 3 Go forwards to the sidecar (5s timeout)
- 4 Sidecar rebuilds the feature vector
- 5 Base models → meta-learner
- 6 Rule engine attaches the recommendation
- 7 JSON returns up the chain

Method	Endpoint	Returns
POST	/api/predict	severity + plan
GET	/api/meta	corridors, zones
GET	/api/locate	station + cluster
GET	/api/health	sidecar status

Models are Python objects **Go cannot load**, so each tier stays in the language it is strongest in. Both are containerised with Docker Compose.

Limitations

- 1 **Single city, single period:** no cross-city generalisation evaluated.
- 2 **Severity is a proxy**, constructed from operational priority and closure fields rather than measured congestion.
- 3 **Critical recall remains low** (0.36 at best), useful as decision support, not safe unsupervised.
- 4 **Stacking underperformed** the base models in this run.
- 5 **The rule engine is not learned:** its manpower figures encode convention, unvalidated against outcome data.

Future scope

- 1 **Rare-class techniques:** SMOTE or focal loss aimed at the Medium and Critical classes.
- 2 **Decision-threshold tuning:** optimise the operating point for recall instead of accepting the default argmax.
- 3 **Class weighting for the neural models:** the single change most likely to lift the MLP's 0.03 Critical recall.
- 4 **Heavier in-fold base models:** test whether that explains the stacking shortfall.
- 5 **Live traffic and weather feeds, and multi-city validation.**

Summary

- ▶ Constructed a four-class severity target from operational fields where the raw log provides none, **removing both source fields** from the features to keep the task honest.
- ▶ Engineered 30 spatial, temporal and contextual features, including unsupervised K-Means location clustering.
- ▶ Trained and compared four heterogeneous classifiers plus an out-of-fold stacked ensemble.
- ▶ **LightGBM is the best model on macro-F1 (0.6949)** despite TabNet posting the highest raw accuracy.
- ▶ Reported the stacking shortfall honestly rather than omitting it.
- ▶ Coupled the prediction to a deterministic recommendation engine and deployed the whole pipeline as a working service.

Central contribution

On imbalanced, safety-relevant traffic data, **accuracy actively misleads**: a 92.4%-accurate model can be 97% blind to the class it exists to catch.

- ▶ G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *NeurIPS*, 2017.
- ▶ T. Chen & C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD*, 2016.
- ▶ S. O. Arik & T. Pfister, "TabNet: Attentive Interpretable Tabular Learning," *AAAI*, 2021.
- ▶ D. H. Wolpert, "Stacked Generalization," *Neural Networks*, 1992.
- ▶ N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," *JAIR*, 2002.
- ▶ D. P. Kingma & J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.
- ▶ S. Iranitalab & A. Khattak, "Comparison of Four Statistical and Machine Learning Methods for Crash Severity Prediction," *Accident Analysis & Prevention*, 2017.
- ▶ M. Zhu et al., "Crash Injury Severity Prediction Using an Ordinal Classification Machine Learning Approach," *IJERPH*, 2021.
- ▶ H. Yin et al., "Deep Learning on Traffic Prediction: Methods, Analysis and Future Directions," *IEEE T-ITS*, 2022.

Thank You

Aman Kumar	2023IMT-010
Sahil Pal	2023IMT-069
Vishwajit Sarak Patil	2023IMT-071

Supervisor: Dr. Saswata Roy
Department of Information Technology, ABV-IIITM Gwalior